MODUL 11 · BAB IV · DYNAMIC PROGRAMMING

Perkalian Rantai Matriks (Matrix-Chain Multiplication)

Menentukan urutan perkalian matriks yang meminimalkan banyaknya operasi perkalian skalar dengan Dynamic Programming.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menerapkan syarat perkalian dua matriks dan menghitung banyaknya operasi perkalian $m \times n \times r$.
- Menjelaskan mengapa urutan penempatan kurung memengaruhi biaya perkalian rantai matriks.
- Menurunkan rumus rekursif $m[i, j]$ dan menjelaskan isi tabel m dan tabel s.
- Menjalankan MATRIX-CHAIN-ORDER dan menyusun urutan perkalian optimal dari tabel s.
- Menjelaskan running time $\Theta(n^3)$ dan memori $\Theta(n^2)$ algoritma ini.

PRASYARAT

- Modul 10: konsep dan empat langkah Dynamic Programming.
- Dasar aljabar matriks (perkalian matriks).

BAB / MATERI ASAL

Materi 8 DYNAMIC PROGRAMMING
Perkalian Rantai Matriks

POSISI DALAM URUTAN BELAJAR

Penerapan pertama Dynamic Programming (Modul 10). Dilanjutkan dengan LCS di Modul 12.

DAFTAR ISI

1. Urutan Mengalikan Ternyata Penting 1
2. Perkalian Dua Matriks 2
3. Masalah Perkalian Rantai Matriks 3
4. Penyelesaian dengan Dynamic Programming 3
5. Contoh Lengkap: Enam Matriks 5
6. Analisis dan Keterkaitan 8
7. Kesimpulan 8
- Kuis Pemahaman 9
- Kunci Jawaban dan Pembahasan 10
- Referensi 10

PENDAHULUAN

1. Urutan Mengalikan Ternyata Penting

Perkalian matriks bersifat **asosiatif**: $(A \times B) \times C = A \times (B \times C)$. Hasilnya sama, apa pun urutan pengelompokannya. Namun **banyaknya pekerjaan** yang dibutuhkan untuk mendapatkan hasil itu bisa sangat berbeda. Pada contoh di modul ini, satu urutan membutuhkan 7.500 operasi perkalian dan urutan lainnya 75.000, sepuluh kali lipat, untuk hasil yang persis sama.

Masalah **perkalian rantai matriks** menanyakan: diberikan barisan matriks $A_1, A_2, \ldots, A_n$, bagaimana menempatkan tanda kurung agar banyaknya operasi perkalian skalar **minimum**? Mencoba semua

kemungkinan tidak praktis karena banyaknya cara tumbuh secara eksponensial. Masalah ini adalah contoh klasik penerapan Dynamic Programming dan memperlihatkan keempat langkahnya secara lengkap.

Modul ini dimulai dari dasar perkalian dua matriks, lalu membangun algoritma **MATRIX-CHAIN-ORDER** langkah demi langkah dan menelusurinya pada contoh enam matriks dari bahan kuliah.

PEMBAHASAN INTI

2. Perkalian Dua Matriks

2.1 Syarat perkalian dan ukuran hasil

Matriks A dan B dapat dikalikan ($A \times B$) apabila **banyaknya kolom A sama dengan banyaknya baris B**. Jika A berukuran $m \times n$ dan B berukuran $n \times r$, hasilnya adalah matriks C berukuran $m \times r$: banyaknya baris mengikuti matriks kiri dan banyaknya kolom mengikuti matriks kanan.

$$A_{m \times n} \times B_{n \times r} = C_{m \times r}$$

CONTOH DARI BAHAN KULIAH

1. A berukuran 5×4, B berukuran 4×8. $A \times B$ dapat dilakukan karena kolom A = baris B = 4. Hasilnya berukuran 5×8.
2. C berukuran 3×2, D berukuran 4×3. $C \times D$ **tidak** dapat dilakukan (kolom C = 2, baris D = 4). Tetapi $D \times C$ dapat dilakukan (kolom D = 3 = baris C), hasilnya 4×2. Perkalian matriks tidak komutatif: urutan matriks tidak boleh ditukar.

2.2 Cara mengalikan dan banyaknya operasi

Elemen baris ke-i kolom ke-j hasil perkalian adalah

$$c_{ij} = \sum_{k=1}^{n} a_{ik}\, b_{kj},$$

yaitu baris ke-i matriks kiri "dikalikan titik" dengan kolom ke-j matriks kanan. Contoh dari bahan kuliah, A berukuran 3×3 dan B berukuran 3×2:

$$\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} \begin{pmatrix} p & q \\ r & s \\ t & u \end{pmatrix} = \begin{pmatrix} ap+br+ct & aq+bs+cu \\ dp+er+ft & dq+es+fu \\ gp+hr+it & gq+hs+iu \end{pmatrix}$$

Setiap elemen hasil membutuhkan 3 perkalian, dan ada $3 \times 2 = 6$ elemen, sehingga totalnya $3 \times 3 \times 2 = 18$ operasi perkalian. Secara umum:

DEFINISI

Mengalikan matriks $m \times n$ dengan matriks $n \times r$ membutuhkan $m \times n \times r$ operasi perkalian skalar: ada $m \cdot r$ elemen hasil, masing-masing membutuhkan n perkalian.

Contoh: A berukuran 20×5 dan B berukuran 5×4 membutuhkan $20 \times 5 \times 4 = 400$ operasi perkalian.

CATATAN KOREKSI TERHADAP SUMBER

Bahan kuliah menjelaskan bahwa komputer menghitung 5×3 dengan cara $3 + 3 + 3 + 3 + 3$. Ini **tidak tepat** untuk prosesor modern, yang memiliki rangkaian pengali (*hardware multiplier*) dan tidak melakukan penjumlahan berulang. Alasan yang tepat mengapa kita menghitung banyaknya perkalian skalar adalah: setiap perkalian skalar pada perkalian matriks berpasangan dengan satu penjumlahan, sehingga banyaknya perkalian skalar sebanding dengan total pekerjaan dan menjadi ukuran biaya yang wajar (Cormen dkk., 2009, Bab 15.2). Kesimpulan bahan kuliah tetap berlaku: masalahnya adalah mencari urutan dengan banyaknya perkalian skalar **paling minimal**.

PEMBAHASAN INTI

3. Masalah Perkalian Rantai Matriks

Diberikan barisan matriks $\langle A_1, A_2, \ldots, A_n \rangle$ yang setiap pasangan berurutannya memenuhi syarat perkalian. **Perkalian rantai matriks** adalah perkalian $A_1 \times A_2 \times \cdots \times A_n$, yang dikerjakan dengan mengalikan dua matriks sekali waktu. Urutannya ditentukan oleh penempatan tanda kurung. Untuk empat matriks ada 5 cara:

$(A_1(A_2(A_3A_4)))$	$(A_1((A_2A_3)A_4))$	$((A_1A_2)(A_3A_4))$	$((A_1(A_2A_3))A_4)$	$(((A_1A_2)A_3)A_4)$

Tabel 1. Lima cara menempatkan kurung pada $A_1A_2A_3A_4$.

3.1 Contoh: tiga matriks

Soal dari bahan kuliah: A_1 berukuran 2×3, A_2 berukuran 3×8, A_3 berukuran 8×5. Berapa operasi perkalian minimal untuk $A_1 \times A_2 \times A_3$?

Cara	Langkah	Operasi
1: $(A_1 \times (A_2 \times A_3))$	$A_2 \times A_3$: $3 \times 8 \times 5 = 120$, hasil B berukuran 3×5. $A_1 \times B$: $2 \times 3 \times 5 = 30$.	$120 + 30 = 150$
2: $((A_1 \times A_2) \times A_3)$	$A_1 \times A_2$: $2 \times 3 \times 8 = 48$, hasil C berukuran 2×8. $C \times A_3$: $2 \times 8 \times 5 = 80$.	$48 + 80 = 128$

Tabel 2. Dua cara mengalikan tiga matriks.

Jumlah operasi minimal adalah **128**, dengan cara 2.

3.2 Mengapa tidak mencoba semua cara?

Banyaknya cara menempatkan kurung pada n matriks, $P(n)$, memenuhi $P(1) = 1$ dan $P(n) = \sum_{k=1}^{n-1} P(k)\,P(n-k)$: pecah di posisi k, lalu kalikan banyaknya cara di sisi kiri dan kanan. Nilainya adalah barisan bilangan Catalan: 1, 1, 2, 5, 14, 42, 132, 429, 1.430, 4.862, ... yang tumbuh paling sedikit secara eksponensial, $\Omega(2^n)$ (Cormen dkk., 2009). Untuk 20 matriks saja ada lebih dari 1,7 miliar cara. Dynamic Programming menyelesaikannya dalam waktu polinomial.

PEMBAHASAN INTI

4. Penyelesaian dengan Dynamic Programming

Kita ikuti empat langkah Dynamic Programming dari Modul 10. Notasi penting: matriks A_i berukuran $p_{i-1} \times p_i$. Seluruh ukuran dapat disimpan dalam satu barisan $p = \langle p_0, p_1, \ldots, p_n \rangle$, dan $A_{i..j}$ menyatakan hasil perkalian $A_i A_{i+1} \cdots A_j$, yang berukuran $p_{i-1} \times p_j$.

4.1 Langkah 1: struktur solusi optimal

Setiap penempatan kurung untuk $A_{i..j}$ (dengan $i < j$) pasti memiliki satu perkalian **terakhir**: memecah rantai di antara A_k dan A_{k+1} untuk suatu $i \le k < j$, yaitu $A_{i..j} = A_{i..k} \times A_{k+1..j}$. Biayanya adalah biaya menghitung $A_{i..k}$, ditambah biaya menghitung $A_{k+1..j}$, ditambah biaya mengalikan keduanya ($p_{i-1}\, p_k\, p_j$). Jika penempatan kurung untuk $A_{i..j}$ optimal, maka penempatan kurung di dalam $A_{i..k}$ dan $A_{k+1..j}$ juga harus optimal (argumen potong-dan-tempel dari Modul 10).

4.2 Langkah 2: rumus rekursif

Misalkan $m[i, j]$ adalah banyaknya operasi perkalian minimum untuk menghitung $A_{i..j}$. Maka

$$m[i,j] = \begin{cases} 0, & \text{jika } i = j, \\ \min\limits_{i \le k < j} \{\, m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j \,\}, & \text{jika } i < j. \end{cases} \qquad (*)$$

Rantai berisi satu matriks tidak membutuhkan perkalian ($m[i, i] = 0$). Untuk rantai yang lebih panjang, coba setiap letak pemecah k dan ambil yang termurah. Nilai k terbaik dicatat di tabel $s[i, j]$.

4.3 Langkah 3: hitung secara bottom-up

Nilai $m[i, j]$ bergantung pada $m[i, k]$ dan $m[k + 1, j]$, yang merupakan rantai lebih **pendek**. Maka tabel diisi berdasarkan **panjang rantai** $l = j - i + 1$: mulai dari $l = 1$ (semua bernilai 0), lalu $l = 2$, $l = 3$, dan seterusnya sampai $l = n$.

MATRIX-CHAIN-ORDER(p)

```
1  n = p.length - 1
2  let m[1..n, 1..n] and s[1..n-1, 2..n] be new tables
3  for i = 1 to n
4      m[i, i] = 0                        // rantai panjang 1
5  for l = 2 to n                         // l adalah panjang rantai
6      for i = 1 to n - l + 1
7          j = i + l - 1
8          m[i, j] = ∞
9          for k = i to j - 1             // coba setiap letak pemecah
10             q = m[i, k] + m[k + 1, j] + p[i-1] * p[k] * p[j]
11             if q < m[i, j]
12                 m[i, j] = q
13                 s[i, j] = k
14 return m and s
```

- **Baris 1–4**: menentukan banyaknya matriks n dari panjang barisan p (yang berisi $n + 1$ bilangan), menyiapkan tabel, dan mengisi diagonal $m[i, i] = 0$.
- **Baris 5–7**: untuk setiap panjang rantai l dan setiap titik awal i, tentukan titik akhir j.

- **Baris 8–13**: mulai dari ∞, coba semua k menurut rumus $(*)$, simpan nilai terkecil di $m[i,j]$ dan letak pemecahnya di $s[i,j]$.

4.4 Langkah 4: susun solusi optimal

$s[i,j]$ menyimpan letak pemecah terakhir yang optimal untuk $A_{i..j}$. Maka $A_{i..j}$ dikalikan sebagai $(A_{i..s[i,j]})(A_{s[i,j]+1..j})$, dan kedua bagiannya dipecah lagi menurut tabel s secara rekursif:

PRINT-OPTIMAL-PARENS(s, i, j) // Cormen dkk., 2009

```
1  if i == j
2      print "A"i
3  else print "("
4      PRINT-OPTIMAL-PARENS(s, i, s[i, j])
5      PRINT-OPTIMAL-PARENS(s, s[i, j] + 1, j)
6      print ")"
```

CONTOH DAN PENERAPAN

5. Contoh Lengkap: Enam Matriks

Contoh dari bahan kuliah (juga contoh baku pada Cormen dkk., 2009):

Matriks	A_1	A_2	A_3	A_4	A_5	A_6
Ukuran	30×35	35×15	15×5	5×10	10×20	20×25

Tabel 3. Ukuran enam matriks.

Barisan ukurannya $p = \langle p_0, \ldots, p_6 \rangle = \langle 30, 35, 15, 5, 10, 20, 25 \rangle$. Rantai ini dapat dipecah pertama kali di $k = 1, 2, 3, 4, 5$, dan setiap bagian yang berisi lebih dari dua matriks harus dipecah lagi. Menghitung semuanya secara manual sangat melelahkan; MATRIX-CHAIN-ORDER menghasilkan tabel berikut.

i \ j	1	2	3	4	5	6
1	0	15.750	7.875	9.375	11.875	15.125
2		0	2.625	4.375	**7.125**	10.500
3			0	750	2.500	5.375
4				0	1.000	3.500
5					0	5.000
6						0

Tabel 4. Tabel $m[i,j]$: operasi minimum untuk $A_i \cdots A_j$.

i \ j	2	3	4	5	6
1	1	1	3	3	3
2		2	3	**3**	3
3			3	3	3
4				4	5
5					5

Tabel 5. Tabel $s[i, j]$: letak pemecah optimal k.

Hanya bagian $i \leq j$ yang terpakai. Tabel diisi per diagonal: diagonal utama ($l = 1$) bernilai 0, lalu diagonal $l = 2$ ($m[1,2], m[2,3], \ldots$), dan seterusnya sampai pojok kanan atas $m[1,6]$ ($l = 6$).

5.1 Menghitung beberapa isi tabel

CONTOH 1: $M[1,2] = 15.750$

Untuk $A_1 \times A_2$ hanya ada satu pilihan, $k = 1$:

$$m[1,2] = m[1,1] + m[2,2] + p_0\, p_1\, p_2 = 0 + 0 + 30 \cdot 35 \cdot 15 = 15.750.$$

CONTOH 2: $M[2,5] = 7.125$ **(DARI BAHAN KULIAH)**

Untuk $A_2A_3A_4A_5$ ada tiga pilihan $k = 2, 3, 4$:

$$m[2,5] = \min \begin{cases} m[2,2] + m[3,5] + p_1p_2p_5 = 0 + 2.500 + 35 \cdot 15 \cdot 20 = 13.000 \\ m[2,3] + m[4,5] + p_1p_3p_5 = 2.625 + 1.000 + 35 \cdot 5 \cdot 20 = 7.125 \\ m[2,4] + m[5,5] + p_1p_4p_5 = 4.375 + 0 + 35 \cdot 10 \cdot 20 = 11.375 \end{cases} = 7.125.$$

Minimum dicapai pada $k = 3$, sehingga $s[2,5] = 3$: urutannya $(A_2A_3)(A_4A_5)$.

CONTOH 3: $M[1,6] = 15.125$ **(JAWABAN AKHIR)**

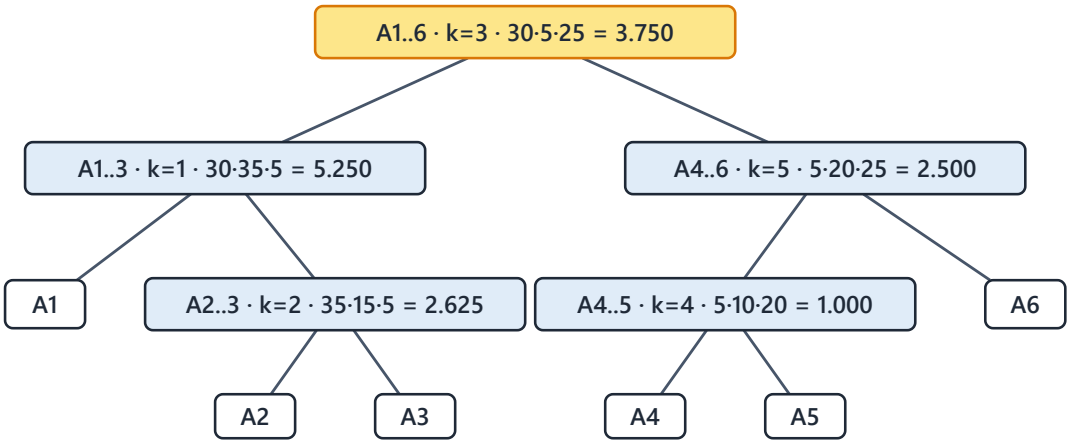
k	$m[1,k]$	$m[k+1,6]$	$p_0\, p_k\, p_6$	**Total**
1	0	10.500	$30 \cdot 35 \cdot 25 = 26.250$	36.750
2	15.750	5.375	$30 \cdot 15 \cdot 25 = 11.250$	32.375
3	7.875	3.500	$30 \cdot 5 \cdot 25 = 3.750$	**15.125**
4	9.375	5.000	$30 \cdot 10 \cdot 25 = 7.500$	21.875
5	11.875	0	$30 \cdot 20 \cdot 25 = 15.000$	26.875

Minimum 15.125 pada $k = 3$, sehingga $s[1,6] = 3$.

5.2 Menyusun urutan perkalian dari tabel s

- $s[1,6] = 3$: rantai dipecah menjadi $(A_1A_2A_3)(A_4A_5A_6)$.
- $s[1,3] = 1$: $A_1A_2A_3$ menjadi $A_1(A_2A_3)$.
- $s[4,6] = 5$: $A_4A_5A_6$ menjadi $(A_4A_5)A_6$.

Hasilnya: $((A_1(A_2A_3))((A_4A_5)A_6))$ dengan banyaknya operasi perkalian minimal $m[1,6] = 15.125$.



Gambar 1. Pohon pemecahan optimal dari tabel s. Setiap simpul dalam adalah satu perkalian dua matriks, ditulis bersama letak pemecah k dan biaya perkalian terakhirnya.

Pemeriksaan: $3.750 + 5.250 + 2.500 + 2.625 + 1.000 = 15.125$. Cocok dengan $m[1,6]$.

5.3 Implementasi Python

Python: MATRIX-CHAIN-ORDER dan PRINT-OPTIMAL-PARENS

```python
import math

def matrix_chain_order(p):
    n = len(p) - 1
    m = [[0] * (n + 1) for _ in range(n + 1)]   # indeks 1..n dipakai
    s = [[0] * (n + 1) for _ in range(n + 1)]
    for l in range(2, n + 1):                   # panjang rantai
        for i in range(1, n - l + 2):
            j = i + l - 1
            m[i][j] = math.inf
            for k in range(i, j):
                q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j]
                if q < m[i][j]:
                    m[i][j], s[i][j] = q, k
    return m, s

def optimal_parens(s, i, j):
    if i == j:
        return f"A{i}"
    k = s[i][j]
    return "(" + optimal_parens(s, i, k) + optimal_parens(s, k + 1, j) + ")"

for p in ([30, 35, 15, 5, 10, 20, 25], [2, 3, 8, 5], [10, 100, 5, 50]):
    m, s = matrix_chain_order(p)
    n = len(p) - 1
    print(m[1][n], optimal_parens(s, 1, n))
```

Keluaran program

```
15125 ((A1(A2A3))((A4A5)A6))
128 ((A1A2)A3)
7500 ((A1A2)A3)
```

ANALISIS DAN HUBUNGAN KONSEP

6. Analisis dan Keterkaitan

6.1 Running time dan memori

MATRIX-CHAIN-ORDER memiliki tiga perulangan bersarang (l, i, k), dan masing-masing berjalan paling banyak $n-1$ kali. Lebih tepatnya, ada $\Theta(n^2)$ pasangan (i, j) dan setiap pasangan mencoba $j-i$ nilai k, sehingga running time-nya $\Theta(n^3)$ (Cormen dkk., 2009). Memori yang dibutuhkan untuk tabel m dan s adalah $\Theta(n^2)$. Dibandingkan brute force yang eksponensial, ini perbaikan yang sangat besar. (Sebagai catatan, ada algoritma yang lebih rumit dengan running time $O(n \lg n)$ dari Hu dan Shing, 1982, tetapi tidak dibahas di sini.)

6.2 Mengapa ini Dynamic Programming, bukan Divide and Conquer?

Rumus $(*)$ dapat ditulis langsung sebagai fungsi rekursif. Namun submasalahnya **tumpang tindih**: misalnya $m[3,4]$ dibutuhkan saat menghitung $m[1,4]$, $m[2,4]$, $m[3,5]$, $m[3,6]$, $m[2,5]$, dan seterusnya. Rekursi biasa akan menghitungnya berkali-kali dan berjalan dalam waktu eksponensial. Tabel m menjamin setiap $m[i,j]$ dihitung tepat sekali.

6.3 Hubungan dengan modul lain

- Ini contoh penerapan keempat langkah Modul 10 dengan sangat jelas: tabel m adalah nilai optimal (langkah 3), tabel s adalah informasi untuk menyusun solusi (langkah 4).
- Pola yang sama (tabel nilai + tabel pilihan) muncul lagi pada LCS (Modul 12) dengan tabel c dan b.
- Running time $\Theta(n^3)$ adalah contoh fungsi kubik dari Modul 05.

KESIMPULAN

7. Kesimpulan

- Matriks $m \times n$ dan $n \times r$ dapat dikalikan bila kolom matriks kiri sama dengan baris matriks kanan; hasilnya $m \times r$ dengan biaya $m \cdot n \cdot r$ perkalian skalar.
- Urutan penempatan kurung tidak mengubah hasil, tetapi dapat mengubah biaya secara drastis. Banyaknya kemungkinan urutan tumbuh eksponensial.
- Rumus rekursif: $m[i,i] = 0$ dan $m[i,j] = \min_{i \le k < j}\{m[i,k] + m[k+1,j] + p_{i-1}p_k p_j\}$.
- MATRIX-CHAIN-ORDER mengisi tabel m dan s berdasarkan panjang rantai, dalam waktu $\Theta(n^3)$ dan memori $\Theta(n^2)$.
- Untuk $p = \langle 30, 35, 15, 5, 10, 20, 25\rangle$, urutan optimalnya $((A_1(A_2A_3))((A_4A_5)A_6))$ dengan 15.125 operasi.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Matriks P berukuran 4×7 dan Q berukuran 7×2. Pernyataan yang benar adalah ...

A. $P \times Q$ berukuran 7×7 dengan 56 operasi perkalian.
B. $P \times Q$ berukuran 4×2 dengan 56 operasi perkalian.
C. $Q \times P$ dapat dilakukan dan berukuran 7×7.
D. $P \times Q$ tidak dapat dilakukan.

2 PERHITUNGAN

A_1 berukuran 10×100, A_2 berukuran 100×5, A_3 berukuran 5×50. Hitung banyaknya operasi untuk $((A_1 A_2) A_3)$ dan $(A_1 (A_2 A_3))$. Mana yang optimal?

3 PERHITUNGAN

Dengan tabel m pada contoh enam matriks, hitung ulang $m[3, 5]$ menggunakan rumus $(*)$ dan tentukan $s[3, 5]$.

4 ANALISIS

Mengapa tabel m harus diisi berdasarkan panjang rantai l, bukan baris demi baris dari $i = 1$?

5 PENYUSUNAN SOLUSI

Dengan tabel s pada contoh enam matriks, tentukan urutan perkalian optimal untuk sub-rantai $A_2 A_3 A_4 A_5 A_6$. Berapa biayanya?

6 PERHITUNGAN

Jalankan MATRIX-CHAIN-ORDER secara manual untuk $p = \langle 4, 10, 3, 12, 20, 7 \rangle$ (lima matriks) sampai diperoleh $m[1, 3]$ dan $s[1, 3]$.

7 BENAR/SALAH DENGAN ALASAN

"Karena ada tiga perulangan bersarang, MATRIX-CHAIN-ORDER membutuhkan memori $\Theta(n^3)$." Benar atau salah?

8 PILIHAN GANDA

Isi $s[i, j]$ menyatakan ...

A. Banyaknya operasi perkalian minimum untuk $A_i \cdots A_j$.
B. Ukuran matriks hasil $A_i \cdots A_j$.
C. Letak pemecah k yang memberikan biaya minimum untuk $A_i \cdots A_j$.
D. Banyaknya cara menempatkan kurung pada $A_i \cdots A_j$.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. Kolom P (7) = baris Q (7), hasil berukuran 4×2, operasi $4 \cdot 7 \cdot 2 = 56$. $Q \times P$ tidak dapat dilakukan karena kolom Q (2) $\neq$ baris P (4).

Soal 2

$((A_1A_2)A_3)$: $10 \cdot 100 \cdot 5 = 5.000$ (hasil 10×5), lalu $10 \cdot 5 \cdot 50 = 2.500$; total **7.500**. $(A_1(A_2A_3))$: $100 \cdot 5 \cdot 50 = 25.000$ (hasil 100×50), lalu $10 \cdot 100 \cdot 50 = 50.000$; total **75.000**. Yang optimal $((A_1A_2)A_3)$, sepuluh kali lebih murah.

Soal 3

$k = 3$: $\quad m[3,3] + m[4,5] + p_2p_3p_5 = 0 + 1.000 + 15 \cdot 5 \cdot 20 = 2.500$. $\quad k = 4$: $\quad m[3,4] + m[5,5] + p_2p_4p_5 = 750 + 0 + 15 \cdot 10 \cdot 20 = 3.750$. Minimum $m[3,5] = 2.500$ dengan $s[3,5] = 3$, yaitu $A_3(A_4A_5)$.

Soal 4

Rumus $m[i,j]$ membutuhkan $m[i,k]$ dan $m[k+1,j]$, yaitu rantai yang lebih pendek. Jika diisi baris demi baris mulai $i = 1$, nilai seperti $m[2,j]$ atau $m[3,j]$ belum tersedia saat menghitung $m[1,j]$. Pengisian berdasarkan panjang rantai menjamin semua rantai yang lebih pendek sudah dihitung.

Soal 5

$s[2,6] = 3$: $(A_2A_3)(A_4A_5A_6)$. $s[2,3] = 2$: (A_2A_3). $s[4,6] = 5$: $((A_4A_5)A_6)$. Urutan: $((A_2A_3)((A_4A_5)A_6))$ dengan biaya $m[2,6] =$ **10.500**. (Cek: $2.625 + 3.500 + 35 \cdot 5 \cdot 25 = 2.625 + 3.500 + 4.375 = 10.500$.)

Soal 6

$m[1,2] = 4 \cdot 10 \cdot 3 = 120$; $m[2,3] = 10 \cdot 3 \cdot 12 = 360$. Untuk $m[1,3]$: $k = 1$: $0 + 360 + 4 \cdot 10 \cdot 12 = 840$; $k = 2$: $120 + 0 + 4 \cdot 3 \cdot 12 = 264$. Jadi $m[1,3] = 264$ dan $s[1,3] = 2$, yaitu $((A_1A_2)A_3)$. (Jika diteruskan sampai akhir, $m[1,5] = 1.344$ dengan urutan $((A_1A_2)((A_3A_4)A_5))$.)

Soal 7

Salah. Tiga perulangan bersarang menentukan **running time** $\Theta(n^3)$. Memori ditentukan oleh ukuran tabel m dan s, masing-masing $n \times n$, sehingga memorinya $\Theta(n^2)$.

Soal 8

Jawaban: C. Pilihan A adalah isi tabel m. Ukuran hasil (B) selalu $p_{i-1} \times p_j$ dan tidak perlu disimpan.

Referensi

Sumber utama: Materi 8 DYNAMIC PROGRAMMING Perkalian Rantai Matriks (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.

2. Hu, T. C., & Shing, M. T. (1982). Computation of Matrix Chain Products. Part I. *SIAM Journal on Computing*, 11(2), 362–373. https://doi.org/10.1137/0211028